02393 Programming in C++

Module 7: Templates

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Templates in C++
- Survey
- Lab

A recap from the previous lectures

- The structure of a C++ program
  - #include and #define directives, the main function, user-defined functions and methods (including constructors, destructors, operators)
- Simple input/output
  - cin, cout
- Variables, values and types
  - string, int, double, float, bool, arrays (statically and dynamically allocated), pointers, enum, struct, vector, ifstream, ofstream, class, this
- Expressions
  - some numeric and boolean operators and math functions, conditional expressions
- Statements
  - if, while, for, switch

Recap on Object-Oriented Programming

- Object-Oriented programming focuses on data:
  - Data types and operations to manipulate them are grouped in classes
  - Classes are the main elements of a program
  - To represent data, classes are instantiated into objects
- A class is similar to a struct, but its members can be:
  - Variables
  - Methods (a bit like functions)
- Class members can be public or private
  - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
  - Constructor: called when an object is created, either statically, or dynamically using new
  - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete
  - Assignment: customizes the behavior of the = operator.

Recap on Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
  - Specify allowed operations on an ADT, by making them public
  - Hide everything else, by making it private
  - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed

The “++” in C++

- So far, we have seen C programming with few elements of C++
  - string
  - cin/cout
  - references
  - vector
- C++ extends C with two key features:
  - Object-Oriented Programming (OOP) (previous lecture)
  - Templates for generic programming (today)

Extending MyVector

- Last time, we implemented our own vector class MyVector
- However, objects of class MyVector can only contain int values
  - What if we need a MyVector of doubles or booleans?
- We need to make our code independent from the data types
- One solution would be to create a copy of MyVector code for each data type and adapt it accordingly
  - Lots of duplicated code, which is not maintainable
  - If a bug is discovered, all variants of MyVector need to be inspected
  - If a new feature is introduced, it must be done for each variant of MyVector

Introducing templates

- Templates: a key feature of C++ enabling generic programming
- Using templates, we can write code generic wrt. some element (types, numbers, etc.)
  – Payoffs: write less duplicated code and simplify maintenance
- The C++ Standard Library provides many facilities based on templates
  – e.g., containers like vector, set, map, etc.

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
  - We indicate that the function is a template

Function templates

```
1   template <class T>
2   T maxF(T a, T b) {
3       if (a < b) {
4           return b;
5       } else {
6           return a;
7       }
8   };
9
10  int main(){
11      int x = maxF<int>(2, 3);
12      double y = maxF<double>(1.2, 3.5);
13  }
```

- To turn a function into a function template:
  - We indicate that the function is a template
  - We specify T as a generic type

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
  - We indicate that the function is a template
  - We specify T as a generic type
  - We replace data types for parameters and/or return value with T

Function templates

```
1   template <class T>
2   T maxF(T a, T b) {
3       if (a < b) {
4           return b;
5       } else {
6           return a;
7       }
8   };
9
10  int main(){
11      int x = maxF<int>(2, 3);
12      double y = maxF<double>(1.2, 3.5);
13  }
```

- To turn a function into a function template:
  - We indicate that the function is a template
  - We specify T as a generic type
  - We replace data types for parameters and/or return value with T
- To instantiate a function template, we specify what data type T will be

Function templates

```
1  struct BankAccount {
2      int number;
3      string owner;
4      double amount;
5  };
6
7  template <class T>
8  T maxF(T a, T b) {
9      if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 template <>
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     double y = maxF<double>(2.2, 3.3);
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile

Function templates

```cpp
struct BankAccount {
    int number;
    string owner;
    double amount;
};

template <class T>
T maxF(T a, T b) {
    if (a < b) {
        return b;
    } else {
        return a;
    }
};

template <>
BankAccount maxF(BankAccount a, BankAccount b) {
    if (a.amount < b.amount) {
        return b;
    } else {
        return a;
    }
};

int main(){
    int x = maxF<int>(2, 3);
    double y = maxF<double>(1.2, 3.5);
    BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
}
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template

Function templates

```
1  struct BankAccount {
2      int number;
3      string owner;
4      double amount;
5  };
6
7  template <class T>
8  T maxF(T a, T b) {
9      if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 template <>
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     double y = maxF<double>(1.2, 3.5);
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template
  - We indicate that the function is a template

Function templates

```
1  struct BankAccount {
2      int number;
3      string owner;
4      double amount;
5  };
6
7  template <class T>
8  T maxF(T a, T b) {
9      if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 template <>
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     double y = maxF<double>(1.2, 3.5);
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template
  - We indicate that the function is a template
  - We specify data types for parameters and/or return value, as we did so far

Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11 template <class A, class B>
12 Pair<A, B>::Pair(A a, B b){
13     this->a = a;
14     this->b = b;
15 }
16 template <class A, class B>
17 A Pair<A, B>::getA(){
18     return this->a;
19 }
20
21 int main(){
22     Pair<int, string> p(10, "foo");
23     cout << p.getA();
24 }
```

- Templates can also be used to define generic classes

Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11 template <class A, class B>
12 Pair<A, B>::Pair(A a, B b){
13     this->a = a;
14     this->b = b;
15 }
16 template <class A, class B>
17 A Pair<A, B>::getA(){
18     return this->a;
19 }
20
21 int main(){
22     Pair<int, string> p(10, "foo");
23     cout << p.getA();
24 }
```

- Templates can also be used to define generic classes
  - When implementing methods, we need to specify that they are also templates

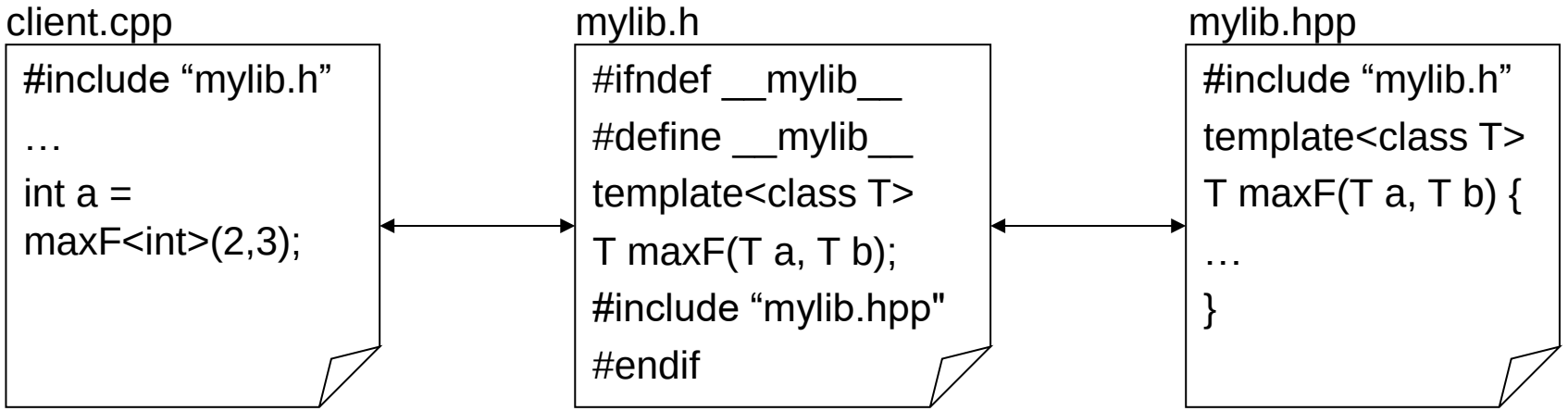
Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11 template <class A, class B>
12 Pair<A, B>::Pair(A a, B b){
13     this->a = a;
14     this->b = b;
15 }
16 template <class A, class B>
17 A Pair<A, B>::getA(){
18     return this->a;
19 }
20
21 int main(){
22     Pair<int, string> p(10, "foo");
23     cout << p.getA();
24 }
```

- Templates can also be used to define generic classes
  - When implementing methods, we need to specify that they are also templates
  - To instantiate a class template, we specify the data types

Templates in libraries

- In previous lectures we have seen that for code reusability we should have:
  - Interfaces (type declarations, function prototypes) in header files (.h)
  - Implementations (function and method definitions) in separate code files (.cpp)
- Template function prototypes go in a header .h file (as expected)
- Template function implementations are a code skeleton (not executable) and can go:
  - In a header .h file (works but it is messy)
  - In a template implementation file .hpp included in the library's main header file



Survey

- Please take part in a brief anonymous midterm survey
- You can find it on https://evaluering.dtu.dk/

Lab time

- Today
  - Make sure C++ works on your computer, request help if it doesn’t
  - Begin working on Assignment 7
    - The deadline is for Tuesday after autumn break
  - Ask questions if something is unclear (including previous assignments)


DTU